Loops and Control Flow Structures

Evan M Drake
drakeev@butte.edu

30/03/2025

Contents

1 Loops and Control Flow

1.1 Loops

1.1.1 theory

What is a loop? Why is it important to computer science (CS)? It is my experience that this topic is usually thrust onto students with the promise that it will make sense one day. Here we will (attempt) to examine a loop with the intent to understand its purpose in CS. See the summation fxn taken from set theory mathematics.

$$\sum_{k=0}^{100} a^k \tag{1}$$

The setup in Eq ((1)) should look familiar, its a basic summation invocation. The intent of the fxn is for all numbers of the form a^k to be summed together. This while very useful to continuous (or simply vast) fields can be a bit difficult to understand. That is, to understand well enough to explain to a computer. We can accomplish this in a variety of ways, though I will tend to usher you towards the way of **iteration**. The art of iteration in computer science is simply to repeat a command over and over again. I like to use the example of the Ouroboros for those trying to visualize it. Iteration will allow us to create repeated work for some arbitrary limit similar to the summation. What I mean by arbitrary limit can be observed as the 100 from Eq ((1)). There is another path to iteration I will introduce briefly but I will not put it here, thus discouraging you from its use.

1.1.2 code

So how do we use iteration in MATLAB? We can do this in a multitude of ways I will illustrate two of them here. The counting method *for* and the waiting method *while*. These are both iterative loop methods that are built into MATLAB. The *for* loop allows for easy counting as you will see below.

```
x = ones(1,10)
for n = 2:6
                    x(n) = 2 * x(n - 1)
end

for <iterator> = <start>:<end>
        <logic using iterator>
end
```

We can see the above code has two forms. The first is a literal MATLAB expression that is take directly from the MATLAB docs[1]. The second is my usual break down of MATLAB syntax. In this case a for loop is stated by using the keyword **for** and defining the iterator. Then one must define the range for the iterator (recall this is discrete space). The next code snippet is dedicated to while loops which instead of counting, set a condition for progression.

```
n = 1;
nFactorial = 1;
while nFactorial < 1e100
                    n = n + 1;
                    nFactorial = nFactorial * n;
end

while <boolean condition>
                    <logic>
end
```

While loops are similar to for loops and while seemingly less complex can often be more difficult to use.

1.2 Control Flow structures

We have already talked about Control Flow in previous lectures. There we also introduced the if statement and a general structure for that as well as

[1] see https://www.mathworks.com/help/matlab/matlab_prog/loop-control-statements.html

the MATLAB structure. Here we will go over some other common structures for conditional logic. The match statement is a way of allowing many choices instead of the standard binary condition. These structures commonly take the form of a mathematical condition structure.

$$x = \begin{cases} 1 & y = 2 \\ 2 & y = 4 \\ y^2 & y \neq 4 \end{cases} \tag{2}$$

We can see in Eq ((2)) that x takes on different value depending on some variable y. This is a common structure in mathematics that does not have the form of an if statement. Thus like most everything else in CS we have taken that general form and made it our own in the form of match (switch) expressions. Below the form is shown in MATLAB.

```
n = input('Enter a number: ');

switch n
    case -1
        disp('negative one')
    case 0
        disp('zero')
    case 1
        disp('positive one')
    otherwise
        disp('other value')
end

switch <match variable>

        case <match value 1>
                <logic for match 1>
                                                .
                                                .
                                                .
        case <match value n>
                <logic for match n>
        otherwise
                <logic for not cases matched>
end
```

2 Applications

We will discuss the application in class, then and only then will I fill this section out.

2.1 theory

2.2 code

Nomenclature

$\neq$	symbol that equates to truth only when the two given values are not the same
$\sum$	The summation function taken from set theory